

Music-Driven DMX Lighting Control System

Blake Stewart, Leo DeRosa, Tom Yasinchuk, Zachary Zhao

Department of Computer Science, University of Victoria, Canada

{blakestewart, lderosa, tomyasinchuk, zackjz}@uvic.ca

ABSTRACT

This report presents the progress of a music-driven DMX lighting control system that maps audio features extracted via Music Information Retrieval (MIR) techniques to real-time lighting parameters. The system extracts rhythmic, spectral, and energy-based features, including tempo, beat strength, RMS energy, onset strength, and spectral centroid and translates them into lighting controls for brightness, colour temperature, and strobe effects using the Open Sound Control (OSC) protocol to communicate with QLC+. A web-based monitoring dashboard was implemented to enable real-time visualization of extracted features and live DMX output values. The current implementation successfully integrates all pipeline components into a functional end-to-end system, achieving all basic performance indicators and most expected performance indicators.

1. INTRODUCTION

Music-driven lighting systems are widely used in live performance, entertainment, and interactive installations. Many existing systems, however, rely on simple amplitude-based triggers, producing visually reactive but musically incoherent behaviour. This project aims to develop a system that uses MIR techniques to extract meaningful musical features and translate them into stable, expressive, and musically coherent lighting changes.

The system is designed to analyze audio recordings offline, generate a pre-computed timeline of lighting commands, and execute those commands in real time against QLC+, a professional open-source lighting control platform. Communication between the Python pipeline and QLC+ is achieved via OSC over UDP, a lightweight and low-latency network protocol.

This report presents the current progress of the system, including implementation details, results, and evaluation of performance indicators.

2. RELATED WORK

Existing work in music-to-lighting systems frequently relies on low-level audio features such as Root Mean Square (RMS) energy for triggering lighting events. While effective for basic synchronization, these approaches lack musical awareness. This project advances that baseline by incorporating higher-level rhythmic and spectral features.

The beat-tracking approach of Ellis [6] uses dynamic programming to align detected onsets with a metrically consistent beat sequence, thereby forming the rhythmic

backbone of lighting synchronization. Onset detection methods described by Bello et al. [7] enable the system to respond to percussive events such as drum hits and driving strobe triggers. The Librosa library [9] provides efficient Python implementations of these methods.

Spectral centroid is used as an approximation of perceptual brightness [5], enabling mapping to warm and cool colour channels. Bogdanov et al.'s Essentia library [10] and PyTorch [11] offer alternative analysis pathways considered for future expansion. The QLC+ platform [12] provides DMX output and OSC input, forming the hardware interface layer. Communication is handled via the OSC 1.0 specification [13]. Datasets including Music4All [15] and GTZAN [16] provide reference material for evaluating feature extraction quality across genres.

3. SYSTEM OVERVIEW

The system follows a four-stage pipeline: (1) audio loading and preprocessing, (2) MIR feature extraction, (3) feature-to-lighting mapping, and (4) real-time DMX output via OSC. An optional web interface provides monitoring and control.

Audio files are loaded using Librosa [9] and resampled to 22,050 Hz as mono signals. Feature extraction produces a set of frame-aligned arrays, RMS, spectral centroid, onset strength, and beat frames, with a hop length of 512 samples, yielding approximately 43 frames per second, or roughly one frame every 23 ms. These features are normalized to [0, 1] and mapped linearly to four DMX channels: brightness (Channel 1), warm colour (Channel 2), cool colour (Channel 3), and strobe (Channel 4).



Figure 1: Four-stage system pipeline.

The system architecture, including all major components and their interactions, is summarized in Table 1.

Component	Technology	Role
Audio loading	Librosa	Load and resample audio files
Feature extraction	Librosa, Numpy	Extract MIR features
Lighting mapping	Numpy	Map features to DMX values
DMX communication	python-osc	Send OSC messages to QLC+

Web server	Flask, Flask-Socket IO	Serve UI, handle uploads
Frontend	HTML, CSS, JavaScript	Upload, monitor, visualize
Lighting platform	QLC+	Receive OSC and output DMX

Table 1: System Architecture

4. IMPLEMENTATION

The implementation of the system is divided into several key components corresponding to each stage of the pipeline. This section describes the design and functionality of the MIR feature extraction module, the feature-to-lighting mapping logic, the OSC communication layer for DMX control, and the overall system integration, including the web-based monitoring interface.

4.1 MIR Feature Extraction (Blake Stewart, Tom Yasinchuk)

The feature extraction module, implemented in `mir/features.py`, provides two functions: `load_audio` and `extract_features`. The `load_audio` function uses `Librosa` to load supported audio formats (e.g., wav, mp3, flac) as mono signals resampled to 22,050 Hz, a standard rate for MIR tasks that balances efficiency and frequency resolution.

The `extract_features` function computes five frame-based features using a hop length of 512 samples (~43 frames per second). RMS energy, computed via `librosa.feature.rms`, represents signal loudness. Spectral centroid, computed using `librosa.feature.spectral_centroid`, captures perceptual brightness, with higher values indicating treble-heavy content. Onset strength, computed using `librosa.onset.onset_strength` with median aggregation, detects transients and percussive events. Tempo estimation and beat tracking are performed using `librosa.beat.beat_track`, returning BPM and beat frame indices. Frame timestamps are computed using `librosa.frames_to_time` to align all features on a common time axis. All extracted features are returned in a single dictionary for use in the lighting mapping stage.

4.2 Feature-to-Lighting Mapping (Leo DeRosa)

The lighting mapping module is implemented in `lighting/controller.py` and translates raw MIR features into DMX channel values. A `normalize_feature` helper function first scales each feature array to the range [0, 1] using min-max normalization, with a guard against near-zero ranges to avoid division errors. This normalization ensures consistent behaviour across different songs regardless of their absolute energy levels.

The `map_features_to_lighting` function iterates over all frames and computes four-channel values. Brightness (Channel 1) is mapped directly from normalized RMS energy, so louder moments produce brighter lights. Warm colour (Channel 2) is mapped inversely from the normalized spectral centroid, meaning bass-heavy content with a low centroid produces more warm output. Cool colour (Channel 3) is mapped directly from the normalized spectral centroid, so high-frequency content produces more cool output. The warm and cool channels are complementary and always sum to approximately 255, creating a dynamic colour temperature that shifts with the tonal character of the music. The strobe (Channel 4) is a boolean trigger that activates when the normalized onset strength exceeds 0.4 and the brightness exceeds 80, ensuring the strobe only fires on strong percussive events when the music is sufficiently loud. The function returns a list of frame dictionaries, each containing a timestamp and the four channel values, which is consumed by the OSC output layer.

4.3 OSC Communication and QLC+ Integration (Leo DeRosa, Blake Stewart)

The `QLCController` class in `lighting/qlc_sender.py` wraps the python-osc `SimpleUDPClient` to provide a clean channel-level interface to QLC+. The controller connects to QLC+ via UDP at 127.0.0.1 on port 7700. Each logical channel is mapped to a named OSC path: channel 1 sends to `/mir/brightness`, channel 2 to `/mir/warm`, channel 3 to `/mir/cool`, and channel 4 to `/mir/strobe`. These named paths correspond to OSC-controlled sliders or widgets configured inside the QLC+ Virtual Console. The `set_channel` method clamps values to the range [0, 255] and sends the raw integer value via the UDP client. A `blackout` convenience method iterates over all four channels and sets each to zero, which is used both at the end of playback and during pause events. Using named OSC paths rather than raw DMX universe/channel addresses makes the QLC+ workspace more readable and allows the fixture mapping to be adjusted inside QLC+ without changing the Python code.

4.4 System Integration and Real-Time Pipeline (Zachary Zhao)

Drag-and-drop or click-to-select file upload supporting .wav, .mp3, .flac, .ogg, and .m4a formats. Real-time progress bar reflecting the processing pipeline stages (loading, extracting, mapping, playing). Audio information panel displaying extracted tempo (BPM), detected beat count, and total duration. Live DMX channel visualization with animated bar graphs for brightness, warm colour, and cool colour channels (0–255), and a pulsing indicator for the strobe channel. Browser-based audio playback synchronized with the DMX command loop. The audio file is loaded as a Blob URL in the browser and played via the HTML5 Audio API, triggered by a WebSocket event emitted by the server at the start of playback. Pause and resume control that simultaneously suspends the DMX

loop on the server side (with an automatic blackout) and pauses the browser audio, then resumes both together. The project environment was established using a Makefile with targets for setup, run, ui, test-osc, test-channels, and clean. Dependencies are tracked in requirements.txt.

4.5 Web-Based Monitoring Dashboard (Zachary Zhao)

A web interface was implemented to satisfy PI5. The backend is a Flask application (UI/server.py) that handles file uploads, processes audio in a background thread, and pushes status updates to connected clients via WebSocket using Flask-SocketIO. The frontend provides drag-and-drop file upload, a real-time progress bar, an audio information panel (tempo, beats, duration), live DMX channel visualization with animated bar graphs, and browser-based audio playback synchronized with the DMX loop via the HTML5 Audio API. A pause/resume control suspends both the DMX loop and browser audio simultaneously.

5. PERFORMANCE INDICATORS AND INDIVIDUAL STATUS REPORTS

This section provides a detailed account of each team member's individual contributions and progress toward their assigned performance indicators (PIs). Each subsection reports the status of basic, expected, and advanced objectives, clearly indicating completed, in-progress, and planned work.

5.1 Blake Stewart - Rhythmic Feature Extraction

PI1 (basic) is complete. Tempo estimation and beat tracking are implemented in mir/features.py using librosa.beat.beat_track, providing BPM estimates and beat frame indices. This establishes the rhythmic backbone for beat-aligned lighting control.

PI2 (basic) is complete. Rhythmic onset activity is extracted using librosa.onset.onset_strength with median aggregation, producing a frame-aligned onset envelope that captures transient and percussive events. This feature is integrated into the pipeline and drives short-duration effects such as strobe triggers.

PI3 (expected) is in progress. The system now includes spectral flux, providing a stronger measure of temporal change. However, beat tracking remains global and does not adapt to local tempo variation. Future work will focus on dynamic or streaming beat tracking to improve alignment between musical events and lighting responses.

PI4 (expected) is in progress. While rhythmic features are correctly extracted, their expressive use is limited by a simplified QLC+ setup. Currently, a single RGBW fixture is used to ensure reliable OSC communication and incremental development. This validates the pipeline but restricts visualization. Future work will expand to multiple

fixtures and scene-based control, enabling richer and more spatially distributed lighting behaviours.

PI5 (advanced) is not yet started. Pulse clarity extraction and multi-band onset detection are planned to enable finer-grained rhythmic analysis (e.g., distinguishing bass vs. percussion) and support more expressive lighting mappings once a more complex QLC+ setup is implemented.

5.2 Leo DeRosa - DMX Mapping Logic

PI1 (basic) is complete. A functional OSC connection between Python and QLC+ is established via the QLCController class in lighting/qlc_sender.py. The connection was verified using the test_osc.py and test_channels.py scripts, which send values to each of the four named OSC paths in QLC+.

PI2 (basic) is complete. RMS energy is normalized and mapped to Channel 1 (brightness), so the overall light intensity responds directly to the energy level of the audio.

PI3 (expected) is complete. Spectral centroid is used to drive the warm and cool colour channels. Low spectral centroid values (bass-heavy content) increase warm output and decrease cool output, while high spectral centroid values (treble-heavy content) do the opposite, creating a dynamic colour temperature that shifts with the tonal character of the music.

PI4 (expected) is complete. A strobe trigger is implemented that fires when normalized onset strength exceeds 0.4 and brightness exceeds 80, restricting strobe activity to strong percussive events during loud passages.

PI5 (advanced) is not yet started. Scene transition logic based on structural change detection remains as planned future work.

5.3 Tom Yasinchuk - Spectral & Structural Analysis

PI1 (basic) is complete. We compute spectral flux as positive differences of L2-normalized magnitude spectra from a single STFT (n_fft=2048, hop_length=512) and expose both frame-wise spectral flux and mean spectral flux. Mean spectral centroid (Hz) is also computed and exposed. These metrics are surfaced to the UI for monitoring.

PI2 (basic) is complete. A silence-detection gate uses an RMS-based relative threshold (2% of max RMS) and a minimum duration of 0.5 s to blackout lights (all channels zero, strobe off) during silent passages. The UI displays a 'Silence Gate' indicator reflecting this state.

PI3 (expected) is not yet started. Harmonic/Percussive Source Separation (HPSS) using librosa.effects.hpss to drive separate light groups from harmonic and percussive

content has been designed and documented, but has not yet been merged into the active pipeline.

PI4 (expected) and PI5 (advanced) are in progress. They are locally undergoing testing until they reach prod status.

5.4 Zachary Zhao - System Integration and Real-Time Pipeline

PI1 (basic) is complete. The project repository was established with a Makefile-based build system, requirements.txt for dependency management, and a virtual environment workflow. Test scripts are included for OSC connectivity and DMX channel verification.

PI2 (basic) is complete. Audio files are loaded using the load_audio function in mir/features.py via Librosa, supporting formats such as wav, mp3, flac, ogg, and m4a. Files are resampled to 22,050 Hz and converted to mono for consistent processing.

PI3 (expected) is complete. A unified control loop is implemented in main/main.py (CLI) and UI/server.py (web interface), integrating feature extraction, lighting mapping, and OSC output into a real-time pipeline that processes audio offline and sends frame-accurate DMX commands during playback.

PI4 (expected) is in progress. Synchronization currently uses time.time() for aligning frame timestamps with real time, providing implicit latency tracking. A dedicated logging system to measure per-frame latency and report timing statistics remains to be implemented.

PI5 (advanced) is complete. A full-stack web interface was developed using Flask and Flask-SocketIO, supporting file uploads, real-time progress tracking, audio metadata display, live DMX visualization, synchronized playback, and pause/resume controls.

6. RESULTS

Testing was conducted using two audio samples: a GTZAN disco track (disco.0.wav) and a longer metal track (Master of Puppets by Metallica).

For the 30.1 s disco sample, the system estimated a tempo of 117.5 BPM and detected 56 beats, close to the expected ~120 BPM. The mean spectral centroid was 2246.4 Hz, and the mean spectral flux was 2.525.

For the 515.4 s metal track, the system estimated a tempo of 103.4 BPM and detected 863 beats, consistent with the track duration. The mean spectral centroid was 3025.7 Hz, and the mean spectral flux was 3.201, indicating higher-frequency content and increased temporal variation.

OSC messages were reliably transmitted to QLC+ (Universe 1) with no observed packet loss. Within QLC+ 5, the mapped DMX channels successfully controlled

RGBW lighting in real time, with outputs responding coherently to musical features, confirming correct end-to-end integration.

The web dashboard rendered correctly across major browsers, with smooth WebSocket updates at a 50 ms interval. Browser-based audio playback exhibited a 200–500 ms delay relative to the DMX loop due to autoplay constraints. Pause/resume functionality worked as expected, with a full blackout during pause.

Overall, the system demonstrates successful integration of feature extraction, communication, and visualization, meeting all basic and most expected objectives.

7. DISCUSSION

The current implementation demonstrates successful end-to-end integration of audio analysis, lighting control, and real-time web monitoring. The system achieves its goal of mapping musical features to lighting parameters in a more expressive and musically coherent manner than simple amplitude-based approaches.

A key challenge was achieving reliable synchronization between audio playback and DMX output. Browser-based audio introduced variable startup latency due to autoplay policies, highlighting the need for centralized timing control. A more robust solution is server-side audio playback with precise timestamping.

Another challenge was the complexity of configuring QLC+, which led to an initial simplified setup using a single RGBW fixture. While this limited expressiveness, it enabled stable communication and incremental development, reinforcing the value of reducing system complexity early on.

A further limitation is the lack of explicit latency measurement. Despite operating at ~23 ms resolution, timing variability may arise from network jitter and Python scheduling. Future work will include implementing per-frame latency logging for more precise evaluation.

Overall, the project demonstrates strong progress and provides valuable experience in integrating signal processing, real-time systems, and interactive visualization.

8. FUTURE WORK GOALS

For the remainder of the project, objectives are planned at three levels: basic (guaranteed), expected (realistic), and advanced (stretch goals). In addition to completing all remaining expected performance indicators, the team will begin implementing selected advanced features.

Basic (guaranteed):

- Implement server-side audio playback using SoundDevice to reduce synchronization delay.

- Refine and stabilize the QLC+ workspace for improved reliability and organization.

Expected (realistic):

- Complete all remaining expected performance indicators.
- Expand QLC+ beyond a single RGBW fixture to support multiple fixtures and scene-based control.
- Implement a latency measurement tool (PI4) to log per-frame timing and report statistics.
- Calibrate lighting mappings across multiple genres and document results.
- Externalize configuration parameters (e.g., QLC+ IP, port, thresholds) to a .env file.

Advanced (stretch):

- Begin remaining advanced indicators, including pulse clarity and multi-band onset detection.
- Add waveform and beat timeline visualizations to the dashboard.
- Explore structural segmentation (e.g., verse/chorus) for scene-level transitions.
- Develop more complex QLC+ scenes and multi-fixture groupings.

While advanced goals are not required at this stage, progress toward them is already underway.

9. CONCLUSION

This report describes the current state of the music-driven DMX lighting control system with a focus on system integration and the web monitoring dashboard. All basic and most expected personal performance indicators have been achieved. The system successfully processes audio files, extracts MIR features, maps them to four-channel DMX lighting parameters, and transmits those parameters to QLC+ in real time. The web interface enables accessible upload, monitoring, and control without requiring command-line interaction. Remaining work focuses on latency measurement, server-side audio playback, and calibration across music genres.

10. REFERENCES

- [1] .G. Tzanetakis, *Music Information Retrieval Book – Companion Code Repository*, GitHub repository, https://github.com/gtzan/mir_book, accessed Feb 2026.
- [2] MusicInformationRetrieval.com contributors, "Symbolic Representations," *Music Information Retrieval*, https://musicinformationretrieval.com/content/2_music_representations/symbolic_representations.html, accessed Feb. 2026.
- [3] Roland, *VC-1-DMX Video Lighting Converter Product Page*, <https://proav.roland.com/global/products/vc-1-dmx>, accessed Feb. 2026.
- [4] MaestroDMX, *MaestroDMX: Intelligent Music-Driven Lighting Controller*, <https://maestrodmx.com>, accessed Feb. 2026.
- [5] D. M. McDermott, "Seeing Harmony, Hearing Color," *Data Science by Design* (blog), <https://datasciencebydesign.org/blog/seeing-harmony-hearing-color>, accessed Feb 2026.
- [6] D. P. W. Ellis, "Beat Tracking by Dynamic Programming," *Journal of New Music Research*, vol. 36, no. 1, pp. 51–60, 2007. <https://doi.org/10.1080/09298210701653344>
- [7] J. P. Bello, L. Daudet, S. Abdallah, C. Duxbury, M. Davies, and M. B. Sandler, "A tutorial on onset detection in music signals," *IEEE Trans. Speech Audio Process.*, vol. 13, no. 5, pp. 1035–1047, Sept. 2005. doi: 10.1109/TSA.2005.851998.
- [8] Python Software Foundation, *Python Documentation – Release 3.14.3*, <https://docs.python.org/3/>, accessed Feb 2026.
- [9] B. McFee, C. Raffel, D. Liang, D. P. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: Audio and music signal analysis in Python," (Version 0.11.0), <https://librosa.org/doc/latest/>, accessed Feb 2026.
- [10] D. Bogdanov, H. Wu, E. Gómez, P. Herrera, O. Mayor, R. Nayak, and X. Serra, "Essentia: A C++ library for audio and music analysis," <https://essentia.upf.edu/>, accessed Feb 2026.
- [11] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," <https://pytorch.org/docs/stable/>, accessed Feb 2026.
- [12] M. Gorski and QLC+ Developers, *QLC+ Documentation*, <https://www.qlcplus.org/docs/>, accessed Feb 2026.
- [13] M. Wright, A. Freed, and A. Momeni, "Open Sound Control 1.0 specification," <https://opensoundcontrol.org/specification/>, accessed Feb 2026.
- [14] W. Howell and Artistic Licence Holdings Ltd., *Art-Net 4 Specification*, <https://artisticlicence.com/WebSiteMaster/ProtocolDownloads/art-net.pdf>, accessed Feb 2026.
- [15] A. Vall, E. Ferraro, and R. Montoliu, "Music4All: A large-scale dataset for music information retrieval," <https://sites.google.com/upf.edu/music4all>, accessed Feb 2026.
- [16] G. Tzanetakis and P. Cook, "Musical genre classification of audio signals," <https://urosevic.net/df/gtzan/>, accessed Feb 2026.